大家好，我是 Guide！到这一步，咱们的“基建三剑客”已经到位了两席：PostgreSQL（向量数据库）和 RustFS（对象存储）。

现在，距离 InterviewGuide 项目起飞只剩下最后一块拼图了——**Redis**。在本项目中，Redis 不仅承担了常规的缓存任务，更通过 **Redis Stream** 实现了简历分析与向量化的**异步解耦**。

话不多说，咱们直接进入最后的一里路：**环境闭环与项目启动！**

Docker 部署 Redis

我们直接拉取 Redis 8.0（拥抱新版本，性能更强）。请注意修改 `-v` 挂载目录为你本地的实际路径。

```
docker run -d \
  --name redis \
  -p 6379:6379 \
  --restart always \
  -v /Users/guide/docker/redis/data:/data \
  redis:8.0 redis-server --appendonly yes
```

各参数说明：

参数	说明
`-d`	**后台运行**（Detached mode），容器启动后在后台默默运行，不占用当前终端窗口。
`--name redis`	**容器自定义名称**，将该容器命名为 `redis`，方便后续管理（如停止、重启、查看日志）。
`-p 6379:6379`	**端口映射**，将宿主机的 `6379` 端口映射到容器内部的 `6379` 端口，允许外部通过本机端口访问 Redis 服务。
`--restart always`	**重启策略**，无论容器是因为错误退出还是 Docker 服务重启（如服务器重启），都会自动重新启动该容器。
`-v /Users/guide/docker/redis/data:/data`	**挂载数据卷**，将宿主机的 `/Users/guide/docker/redis/data` 目录挂载到容器内的 `/data` 目录，实现数据持久化，防止容器删除后数据丢失。
`redis:8.0`	**指定镜像**，使用 Redis 的 8.0 版本镜像启动容器。
`redis-server --appendonly yes`	**启动命令**，容器启动后执行的命令。这里显式启动 Redis 服务并开启 **AOF 持久化**（Append Only File），以提供更高的数据安全性。

本地运行项目

1. 克隆项目

```
# Github
git clone https://github.com/Snailclimb/interview-guide.git
或者
# Gitee
git clone https://gitee.com/SnailClimb/interview-guide.git
cd interview-guide
```

2. 配置数据库

```
-- 创建数据库
CREATE DATABASE interview_guide;

-- 连接数据库并启用 pgvector 扩展（可选，启动后端SpringAI框架底层会自动创建）
CREATE EXTENSION vector;
```

3. 配置环境变量

```
# AI API 密钥（阿里云 DashScope）
export AI_BAILIAN_API_KEY=your_api_key
```

4. 修改应用配置

编辑 `app/src/main/resources/application.yml`：

```
spring:
  # PostgreSQL数据库配置
  datasource:
    url:
jdbc:postgresql://${POSTGRES_HOST:localhost}:${POSTGRES_PORT:5432}/${POSTGRES_DB
:interview_guide}
    username: ${POSTGRES_USER:postgres}
    password: ${POSTGRES_PASSWORD:123456}
    driver-class-name: org.postgresql.Driver

  jpa:
    hibernate:
      ddl-auto: create #首次启动用 create，表创建成功后，改回 update

  # Redisson配置 (使用 spring.redis.redisson，参考官方文档)
  redis:
    redisson:
      config: |
        singleServerConfig:
          address: "redis://${REDIS_HOST:localhost}:${REDIS_PORT:6379}"
          database: 0
          connectionMinimumIdleSize: 10
          connectionPoolSize: 64
          subscriptionConnectionMinimumIdleSize: 1
          subscriptionConnectionPoolSize: 50

# RustFS (S3兼容) 存储配置
app:
  storage:
    endpoint: ${APP_STORAGE_ENDPOINT:http://localhost:9000}
```

```
    access-key: ${APP_STORAGE_ACCESS_KEY:wr45VXJZhCxc6FAWzOYR}
    secret-key:
${APP_STORAGE_SECRET_KEY:GtKxV57WJkpw4CvASPBzTy2DYElLnRqh8dIXQaOm}
    bucket: ${APP_STORAGE_BUCKET:interview-guide}
    region: ${APP_STORAGE_REGION:us-east-1}
```

⚠**注意**:

1. `ddl-auto: create`：首次启动项目时，一定要设置为 `create`，这样 Hibernate 才会根据实体类自动建表。表建好后，**一定要记得改回** `update`，否则下次启动你的数据会被一键清空！
2. **环境变量注入**： 为了安全，`AI_BAILIAN_API_KEY`（通义千问 API 密钥）建议通过系统环境变量注入，不要直接写死在配置文件里。
3. **Minio 兼容**：如果本地已经 Minio 的话，可以用其替换 RusfFS。

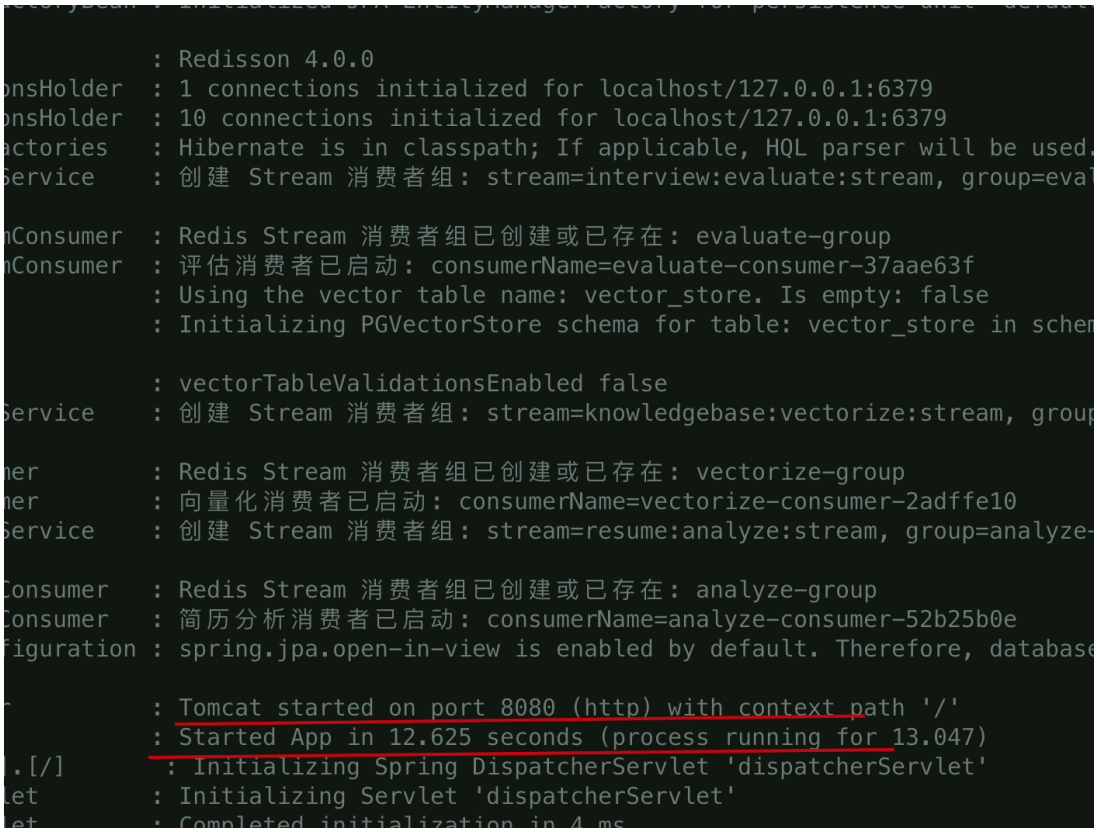
5. 启动服务

后端:

我们使用 Gradle 进行构建:

```
./gradlew bootRun
```

当你在控制台看到 `Started App in XX seconds` 且没有 Redisson 连接报错时，说明后端已经成功连接到了 PostgreSQL 和 Redis。



后端服务启动于 `http://localhost:8080` 。

前端:

进入前端目录，推荐使用 `pnpm`（比 npm 更快）：

```
cd frontend
pnpm install
pnpm dev
```

前端服务启动于 `http://localhost:5173`

总结

到这里，恭喜你，**InterviewGuide 的完整环境已经全部跑通了！**

回顾一下咱们这几篇文章的成果：

1. **PostgreSQL + pgvector**：搭建好了 AI 的“长期记忆”。
2. **RustFS**：配置好了高性能的“储物间”。
3. **Spring AI + 阿里云百炼**：打通了 AI 的“大脑”。
4. **Redis**：构建了任务流转的“中枢神经”。

环境搭建只是“万里长征”的第一步，加油！